ENSET Mohammedia
Université Hassan II de Casablanca

Master SDIA – Systèmes Distribués et Intelligence Artificielle

Sécurisation d'une API REST avec Spring Security 6 & JWT

Nom et prénom :
Ayoub ELHOUANI

Enseignant :
Pr. Abdelmajid
BOUSSELHAM

14 octobre 2026

Table des matières

1 Architecture Globale & Authentification Stateless **3**
1.1 Objectif & Problématique 3
1.2 Schéma d'Architecture Globale en Couches 3

2 Configuration, Modélisation JPA & DTOs **3**
2.1 Configuration de l'Environnement 3
2.2 Modélisation de l'Entité `AppUser` et Table `users` 3
2.3 Couche d'Accès aux Données (`UserRepository`) 4
2.4 Objets de Transfert de Données (DTOs) 5

3 Service Cryptographique JWT (`JwtService`) **5**
3.1 Anatomie d'un Jeton JWT 5
3.2 Implémentation du Service `JwtService` 5
3.3 Synthèse des Méthodes Cryptographiques 6

4 Gestion des Utilisateurs & Spring Security **6**
4.1 Intégration du Contrat Spring Security 6

5 Chaîne de Filtres et Pipeline d'Authentification **7**
5.1 Diagrammes de Séquence des Flux Sécurisés 7
5.2 Configuration de Sécurité (`SecurityFilterChain`) 8

6 Spécification des Endpoints, Tests et Validation **9**
6.1 Matrice des Endpoints de l'API 9
6.2 Validation de la Compilation et Requête Postman 9

7 Sécurité Avancée, Révocation & Scalabilité **10**
7.1 Refresh Tokens & Révocation avec Cache Redis 10
7.2 Signature Symétrique (HMAC) vs Asymétrique (RSA) 10

Conclusion **11**

Table des figures

1 Architecture globale en couches du projet Spring Boot avec JWT 3
2 Schéma de la table de persistance `users` 4
3 Anatomie détaillée d'un jeton JSON Web Token (JWT) 5
4 Cinématique d'authentification et émission du jeton 7
5 Cinématique d'interception et validation du jeton par le filtre 8
6 Exemple de requête d'authentification et obtention du jeton Bearer 10
7 Architecture avec Refresh Token et Blacklisting via Redis 10

1 Architecture Globale & Authentification Stateless

1.1 Objectif & Problématique

L'approche **JWT (RFC 7519)** permet de déporter la preuve d'identité chez le client dans un modèle sans état (*Stateless*). Contrairement à l'authentification avec état (*Stateful*) basée sur les sessions HTTP en mémoire serveur (`JSESSIONID`) qui entrave le passage à l'échelle horizontal (*horizontal scalability*), chaque requête est auto-suffisante et validée cryptographiquement à la volée par les filtres de sécurité.

1.2 Schéma d'Architecture Globale en Couches

La figure 1 synthétise l'architecture en couches mise en place dans notre projet.

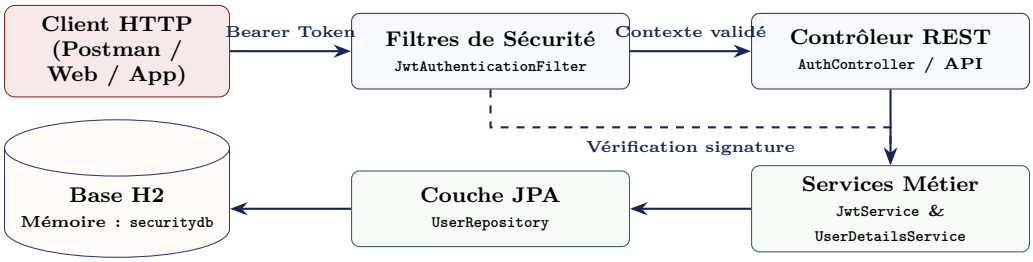


FIGURE 1 – Architecture globale en couches du projet Spring Boot avec JWT

2 Configuration, Modélisation JPA & DTOs

2.1 Configuration de l'Environnement

Le projet est architecturé autour de Java 21 LTS et Spring Boot. Le fichier `application.properties` initialise la base H2 en mémoire et définit les paramètres du jeton JWT :

Listing 1 – Paramètres de persistance et JWT (application.properties)

```
spring.application.name=tp-jwt-spring

# Base de donnees H2 en memoire
spring.datasource.url=jdbc:h2:mem:securitydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# Proprietes JWT (Cle secrete HMAC Base64 et duree d'expiration 1 heure)
jwt.secret=VGHpcv1pcy1hLXZJcnktbG9uZy1zZWNyZXQtaW5n30tZGVtby0MDI2
jwt.expiration=3600000
```

2.2 Modélisation de l'Entité `AppUser` et Table `users`

L'entité `AppUser` modélise l'utilisateur du système. L'adresse email est déclarée unique afin de servir d'identifiant d'authentification.

TABLE RELATIONNELLE : users

- `id` : **BIGINT** [Primary Key, Auto-Increment]
- `email` : **VARCHAR(255)** [UNIQUE, NOT NULL]
- `password` : **VARCHAR(255)** [Hash BCrypt]
- `role` : **VARCHAR(50)** [ROLE_USER, ROLE_ADMIN]

FIGURE 2 – Schéma de la table de persistance `users`

Listing 2 – Entité JPA AppUser (src/.../entity/AppUser.java)

```
package org.example.tpjwtspring.entity;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Table(name = "users")
@Data
public class AppUser {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String email;

    private String password;
    private String role;

    public AppUser() {}
}
```

2.3 Couche d'Accès aux Données (`UserRepository`)

L'interface `UserRepository` étend `JpaRepository`. Spring Data JPA génère dynamiquement les requêtes de recherche et d'existence par email :

Listing 3 – Interface UserRepository (src/.../repo/UserRepository.java)

```
package org.example.tpjwtspring.repo;

import org.example.tpjwtspring.entity.AppUser;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<AppUser, Long> {
    Optional<AppUser> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

2.4 Objets de Transfert de Données (DTOs)

Les données en transit sont modélisées par des **Records Java**, garantissant immuabilité native, légèreté et lisibilité :

Listing 4 – DTOs d'authentification en Records Java

```
package org.example.tpjwtspring.dto;

public record LoginRequest(String email, String pass) {}
public record RegisterRequest(String email, String pass) {}
public record AuthResponse(String token) {}
```

3 Service Cryptographique JWT (JwtService)

3.1 Anatomie d'un Jeton JWT

Un token JWT (RFC 7519) comporte trois composants encodés en *Base64URL* reliés par des points :

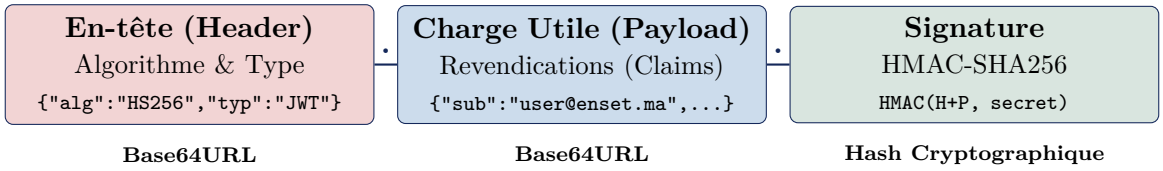


FIGURE 3 – Anatomie détaillée d'un jeton JSON Web Token (JWT)

3.2 Implémentation du Service JwtService

La classe JwtService regroupe la génération, l'extraction et la validation cryptographique des jetons avec l'API moderne de JJWT (0.13.0) :

Listing 5 – Service de gestion des tokens (src/.../service/JwtService.java)

```
package org.example.tpjwtspring.service;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;
import javax.crypto.SecretKey;
import java.util.Date;

@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private long expiration;

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secret);
        return Keys.hmacShaKeyFor(keyBytes);
    }
```

```
26
27      public String generateToken(UserDetails userDetails) {
28          return Jwts.builder()
29                  .subject(userDetails.getUsername())
30                  .issuedAt(new Date())
31                  .expiration(new Date(System.currentTimeMillis() + expiration))
32                  .signWith(getSigningKey())
33                  .compact();
34      }
35
36      public String extractUsername(String token) {
37          return extractAllClaims(token).getSubject();
38      }
39
40      private Claims extractAllClaims(String token) {
41          return Jwts.parser()
42                  .verifyWith(getSigningKey())
43                  .build()
44                  .parseSignedClaims(token)
45                  .getPayload();
46      }
47
48      public boolean isTokenValid(String token, UserDetails userDetails) {
49          final String username = extractUsername(token);
50          return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
51      }
52
53      private boolean isTokenExpired(String token) {
54          return extractAllClaims(token).getExpiration().before(new Date());
55      }
56  }
```

3.3 Synthèse des Méthodes Cryptographiques

Méthode	Retour	Rôle et Opération
`getSigningKey()`	`SecretKey`	Décode la clé Base64 et instancie la clé HMAC-SHA-256.
`generateToken(UserDetai`	`String`	Injecte `sub`, `iat`, `exp` et signe le jeton.
`extractUsername(String)`	`String`	Récupère l'identifiant utilisateur (claim `sub`).
`extractAllClaims(String`	`Claims`	Valide la signature HMAC et extrait le corps JSON.
`isTokenValid(token, user)`	`boolean`	Contrôle l'identité et la validité temporelle.
`isTokenExpired(String)`	`boolean`	Vérifie si l'instant actuel dépasse le timestamp `exp`.

TABLE 1 – Matrice fonctionnelle des méthodes de `JwtService`

4 Gestion des Utilisateurs & Spring Security

4.1 Intégration du Contrat Spring Security

Pour charger les données d'identification lors de la vérification, la classe `CustomUserDetailsService` implémente l'interface de base `UserDetailsService` :

Listing 6 – Service de chargement des identités (CustomUserDetailsService.java)

```java
package org.example.tpjwtspring.service;

import org.example.tpjwtspring.entity.AppUser;
import org.example.tpjwtspring.repo.UserRepository;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username)
            throws UsernameNotFoundException {

        AppUser user = userRepository
                .findByEmail(username)
                .orElseThrow(() ->
                    new UsernameNotFoundException("Utilisateur non trouve : " + username)
                );

        return User.builder()
                .username(user.getEmail())
                .password(user.getPassword())
                .roles(user.getRole())
                .build();
    }
}
```

5 Chaîne de Filtres et Pipeline d'Authentification

5.1 Diagrammes de Séquence des Flux Sécurisés

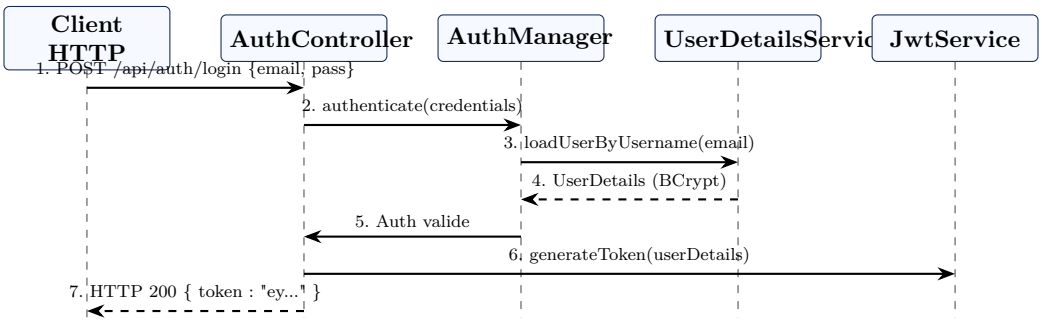


FIGURE 4 – Cinématique d'authentification et émission du jeton

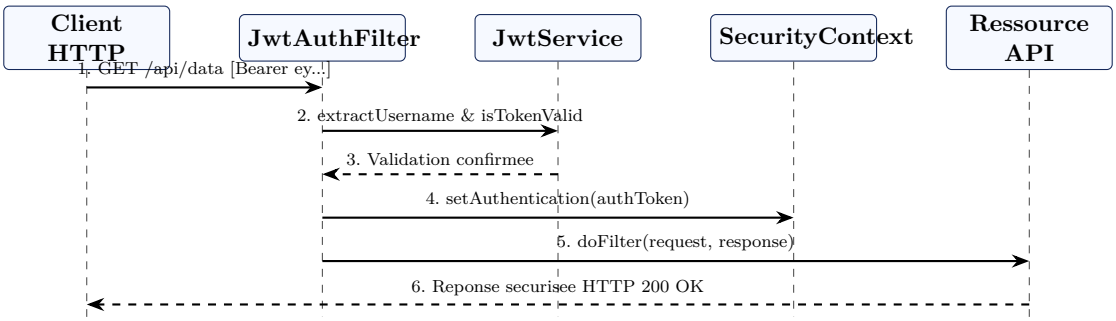


FIGURE 5 – Cinématique d'interception et validation du jeton par le filtre

5.2 Configuration de Sécurité (`SecurityFilterChain`)

Dans Spring Security 6, la configuration est déclarée via un bean `SecurityFilterChain` :

Listing 7 – Configuration de sécurité stateless (SecurityConfig)

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;
    private final CustomUserDetailsService userDetailsService;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter,
                          CustomUserDetailsService userDetailsService) {
        this.jwtAuthFilter = jwtAuthFilter;
        this.userDetailsService = userDetailsService;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable()) // Desactive car pas de session ni cookies
            .sessionManagement(sm ->
                sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**", "/h2-console/**").permitAll()
                .anyRequest().authenticated()
            )
            .headers(h -> h.frameOptions(fo -> fo.disable())) // Console H2
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

6 Spécification des Endpoints, Tests et Validation

Exemples de commandes utilisées

Commande	Description
Commandes de Build Maven	
`./mvnw test-compile`	Compile les sources et vérifie les dépendances.
`./mvnw spring-boot:run`	Lance l'application sur le port local `8080`.
`./mvnw clean package`	Construit l'archive exécutable JAR.
Commandes d'Appel HTTP (cURL)	
`curl -X POST /api/auth/register`	Inscription d'un nouvel utilisateur avec JSON `{email, pass}`.
`curl -X POST /api/auth/login`	Connexion et récupération du jeton JWT.
`curl -H "Authorization: Bearer <T>"`	Requête authentifiée vers une ressource sécurisée.

Table 2 – Commandes utilisées durant le TP

6.1 Matrice des Endpoints de l'API

Méthode	URI Endpoint	En-tête Requise	Payload Requis	Statut
`POST`	`/api/auth/register`	`Content-Type: app/json`	`{email, pass}`	200 / 201
`POST`	`/api/auth/login`	`Content-Type: app/json`	`{email, pass}`	200 OK
`GET`	`/api/user/profile`	`Authorization: Bearer <T>`	Aucun	200 OK
`GET`	`/h2-console`	Authentification web	Identifiants H2	200 OK

Table 3 – Spécification des endpoints REST de l'API d'authentification

6.2 Validation de la Compilation et Requête Postman

Listing 8 – Compilation sans erreur du projet avec Maven Wrapper

```
$ ./mvnw test-compile
[INFO] Scanning for projects...
[INFO] ----------------------< org.example:tp-jwt-spring >-----------------------
[INFO] Building tp-jwt-spring 0.0.1-SNAPSHOT
[INFO] Compiling 8 source files with javac [debug parameters release 21]
[INFO] Compiling 1 source file with javac to target/test-classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] Total time: 10.963 s
[INFO] ------------------------------------------------------------------------
```

```
POST /api/auth/login HTTP/1.1
Host: localhost:8080
Content-Type: application/json

{
"email": "ayoub.elhouani@enset-media.ac.ma",
"pass": "SecretP@ssw0rd2026"
}

HTTP/1.1 200 OK
Content-Type: application/json

{
"token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJheW91Yi5lbGhvdWFuaU...
...NzYwNDc2MDAwLCJleHAiOjE3NjA0Nzk2MDB9.7q0_Z_w9mK18a..."
}
```

FIGURE 6 – Exemple de requête d'authentification et obtention du jeton Bearer

7 Sécurité Avancée, Révocation & Scalabilité

7.1 Refresh Tokens & Révocation avec Cache Redis

L'inconvénient fondamental d'un token JWT sans état est l'impossibilité de l'invalider avant son terme s'il est dérobé. La solution industrielle consiste à employer un **Access Token** à durée courte (15 minutes) couplé à un **Refresh Token** à durée longue (7 jours) stocké dans un cookie `HttpOnly` et révocable via une liste noire en cache mémoire **Redis** :

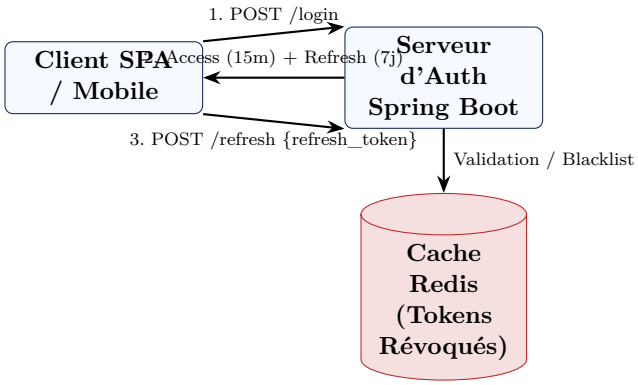


FIGURE 7 – Architecture avec Refresh Token et Blacklisting via Redis

7.2 Signature Symétrique (HMAC) vs Asymétrique (RSA)

Pour des microservices d'envergure, l'algorithme asymétrique **RS256** (RSA avec SHA-256) permet au serveur d'authentification de détenir la **clé privée** pour émettre les jetons, tandis que les microservices tiers vérifient la signature avec la seule **clé publique**, sans risque de compromission de la clé maîtresse.

Conclusion

Ce travail pratique a permis de consolider la mise en place d'une architecture d'authentification moderne et sans état (*Stateless*) sous Spring Boot et Spring Security 6 avec la bibliothèque JJWT.

Le découplage des responsabilités entre le modèle JPA `AppUser`, la couche d'accès aux données `UserRepository`, le chargement des identités (`CustomUserDetailsService`) et la signature des jetons (`JwtService`) garantit une modularité rigoureuse, une haute sécurité et une scalabilité optimale.